

Knowledge Graph Link Prediction with Deep Knowledge Graph Embedding Models

Thalia Delhaise student no. 66937

Lenny Briclet student no. ...

André Fonseca student no. ...

Rafael Gufler student no. ...

Deep Learning & Knowledge Graphs — unified project

Problem and motivation. A *knowledge graph* collects facts as triples (h, r, t) : head entity h , relation r , tail entity t . *Link prediction* answers queries $(h, r, ?)$ or $(?, r, t)$: given the training facts, which missing endpoint is most plausible? We follow standard knowledge graph embedding [2]: learn a score $f_\theta(h, r, t) \in \mathbb{R}$ so true triples rank above corrupted ones.

Pipeline (build the KG, then predict). The project matches both courses in order: **(1) Dataset $\rightarrow$ graph.** We use the public **FB15k-237** benchmark [3]—released as **real triple files** (train/validation/test splits). That release *is* our dataset: thousands of facts about real-world entities, not a toy hand-drawn sketch. **Building** the KG means ingesting those triples, mapping strings to entity/relation IDs, forming the edge set $\mathcal{T}$ and the split, and loading the result into PyKEEN [1]; we may also export a small RDF or edge-list slice for figures or the KG report. **(2) Prediction.** We then train models only on $\mathcal{T}_{\text{train}}$ and evaluate on held-out test triples (next paragraph). We do not invent a private ontology from scratch, but we still **materialize** the full multi-relational graph the methods run on.

What the graph looks like. Nodes are entities; each fact is a **directed, labeled** link $h \xrightarrow{r} t$. Many relation types exist, and the same pair (h, t) can appear under different r . Equivalently, $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$ with $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$. We keep the official benchmark partition unchanged (no extra subsampling).

Task and evaluation protocol. Training uses only $\mathcal{T}_{\text{train}}$: we minimize a **margin/ranking loss** and draw **negative** triples by corrupting head or tail (standard pipeline [2, 3]). At test time, each $(h, r, t) \in \mathcal{T}_{\text{test}}$ was unseen during training; we score all candidate completions and record the rank of the true entity. **Filtered** metrics [2]: when ranking the true tail for $(h, r, ?)$, any other tail t' with $(h, r, t') \in \mathcal{T}_{\text{train}} \cup \mathcal{T}_{\text{val}} \cup \mathcal{T}_{\text{test}}$ is treated as ambiguous (not penalized as an error); the same for head prediction. We average results over head and tail prediction.

Scale. FB15k-237 (Freebase-derived) counts: **14,541** entities, **237** relations, **272,115** train / **17,535** validation / **20,466** test triples—large enough to be meaningful, still feasible on Colab/GPU. An optional second graph (e.g. OGB link prediction) is possible; FB15k-237 remains the main result.

Method (planned). **Baseline:** TRANSE [2]. **Stronger model:** ROTATE [3]. Both via PyKEEN [1] (same splits, negative sampling, and evaluator for fair comparison). We will record embedding dimension d , loss, optimizer, batch size, number of epochs (or early stopping on validation MRR), learning rate, seed, and GPU/CPU time. Course material fills in background reading.

Metrics and qualitative analysis. **Quantitative:** mean reciprocal rank (MRR) and Hits@1, Hits@3, Hits@10 under the filtered protocol, reported for head and tail (then averaged, as is standard). **Qualitative:** loss and validation MRR (or Hits@10) vs. epoch, plus a short look at a few test queries and their top-ranked entities.

References

- [1] M. Ali, M. Berrendorf, C. T. Hoyt, L. Vermue, S. Sharifzadeh, V. Tresp, and J. Lehmann. PyKEEN 1.0: A python library for training and evaluating knowledge graph embeddings. *Journal of Machine Learning Research*, 22(82):1–6, 2021. [JMLR page \(PDF\)](#); [documentation](#).
- [2] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, and O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, 2013. [NeurIPS proceedings \(PDF\)](#).
- [3] Z. Sun, Z.-H. Deng, J.-Y. Nie, and J. Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*, 2019. [OpenReview \(paper + PDF\)](#).